Gerência de configuração

Revisão pra prova

Prof. Diego Cardoso Borda Castro

Comandos git

- git config
  - git config --global alias.ac "!git add . && git commit -m"
  - --system, --global, --project
- git init
- git status
- git add
- git reset --soft HEAD~1
- git rm -r --cached .
- git commit -a -m “meu primeiro commit”

Comandos git

Comando	Commit	Stage	Arquivos locais
git reset --soft HEAD~1	desfaz	mantém	mantém
git reset --mixed HEAD~1	desfaz	remove do stage	mantém
git reset --hard HEAD~1	desfaz	remove do stage	descarta alterações

Comando	O que remove	Remove arquivos ignorados?	Simulação
git clean -n	Mostra arquivos untracked que seriam removidos	Não	Sim
git clean -f	Remove arquivos untracked	Não	Não
git clean -fd	Remove arquivos e diretórios untracked	Não	Não
git clean -nd	Mostra arquivos e diretórios que seriam removidos	Não	Sim
git clean -x	Considera também arquivos ignorados	Sim	—
git clean -fx	Remove arquivos untracked + ignorados	Sim	Não
git clean -fdx	Remove arquivos + diretórios untracked + ignorados	Sim	Não
git clean -ndx	Mostra tudo que seria removido, inclusive ignorados	Sim	Sim

Comandos git

- git log
  - git log -p nome_do_arquivo
- git checkout 99f7c7fb3082937e85c706
- git ignore
- git branch -m feature/teste
- git merge nome-da-branch -m "mensagem do merge"
- git rebase feature/nova
- git tag -a v2.0 -m “nova tag”

Comandos git

- git blame
- git stash
- git push -u origin main
- git remote add origin2 url
- git clone url
- git pull
- git fetch
- git cherry pick

- pull request
- git flow
- fork
- issue

Comandos docker

- Objetos comuns
  - container
  - image
  - volume
  - network
- Ações comuns
  - ls → listar
  - rm → remover
  - run → executar
  - create → criar
  - start / stop → iniciar/parar

docker [objeto] [ação]

Comandos docker

- Container
  - -d (detached)
  - -p (porta)
  - --name
  - -e (var de ambiente)
  - -v (mapeia volume)
  - --rm (remove ao parar)
  - -a (tudo)
  - -i (modo interativo)
  - -t (terminal TTY)
  - -q (mostra só os IDs)

docker [objeto] [ação] [params]

Comandos docker

- Image
  - -t (tag da imagem)
  - -f (Dockerfile custom)
  - --no-cache (build sem cache)
  - --build-arg (passa variáveis pro build)
  - --target (multi-stage)
  - rm
    - -f (força remoção)

docker [objeto] [ação] [params]

Comandos docker

- Volume
  - -d (define o driver. Padrão: local)
  - --label (adiciona metadados ao volume)
  - -q (mostra apenas IDs)
- -f, --filter
  - docker volume ls -f dangling=true

docker [objeto] [ação] [params]

Comandos docker

- Network
  - -d / --driver (tipo da rede)
  - --subnet (define faixa de IP)
  - --gateway
  - --attachable (permite containers overlay)
- docker network connect minha-rede container_id
  - disconnect

docker [objeto] [ação] [params]

Comandos docker

Tipo Comando	Flags	Observação
ls	-q	mostra só os IDs
ls	-f	realiza filtro
rm	-f	força remoção
prune	-f	não pede confirmação
inspect	-f	especifica o formato
inspect	-s	mostra o tamanho

Dockerfile

- FROM
- RUN
- WORKDIR
- COPY
- ADD
- LABEL
- ENV
- VOLUME
- ARG
- EXPOSE
- USER
- CMD
- ENTRYPOINT
- HEALTHCHECK
- SHELL
- ONBUILD

```
1  ARG TAG=latest
2  FROM node:$TAG
3  WORKDIR /app
4  COPY package*.json ./
5  RUN npm i
6  COPY . .
7  EXPOSE 3000
8  CMD ["node", "src/index"]
```

Docker compose

- docker compose ls
- docker compose logs
- docker compose logs service name
- docker compose exec service name
- docker compose up
- docker compose start
- docker compose stop
- docker compose down
- docker compose ps
- docker compose build
- docker compose build app
- docker compose restart
- docker compose restart app
- docker compose pull
- docker compose rm app
- docker compose kill app

Docker compose

- services
- container_name
- image
- ports
- command
- environment
- volumes
- networks
- build
- extra_hosts
- network_mode
- context
- dockerfile
- Vários compose
  - Extends
  - Merge
  - Include
- profiles
- restart
- env_file
- healthcheck
- depends_on
  - condition
- secrets
- configs
- logging
- stdin_open / tty

Docker compose

```yaml
services:
  postgres:
    container_name: postgres
    image: postgres:12.17
    ports:
      - "5432:5432"
    environment:
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=postgres
      - POSTGRES_DB=postgres
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - postgres_network
```

```yaml
  pgadmin:
    container_name: pgadmin
    image: dpage/pgadmin4:latest
    ports:
      - "8080:80"
    environment:
      - PGADMIN_DEFAULT_EMAIL=admin@admin.com
      - PGADMIN_DEFAULT_PASSWORD=admin
    depends_on:
      - postgres
    networks:
      - postgres_network

volumes:
  postgres_data:
    name: postgres_data
    external: true

networks:
  postgres_network:
    driver: bridge
```

Docker compose

- Variáveis de ambiente
  - POSTGRES_VERSION=17 docker compose up -d
  - docker compose config
  - docker compose up -d --env-file=arquivo.env

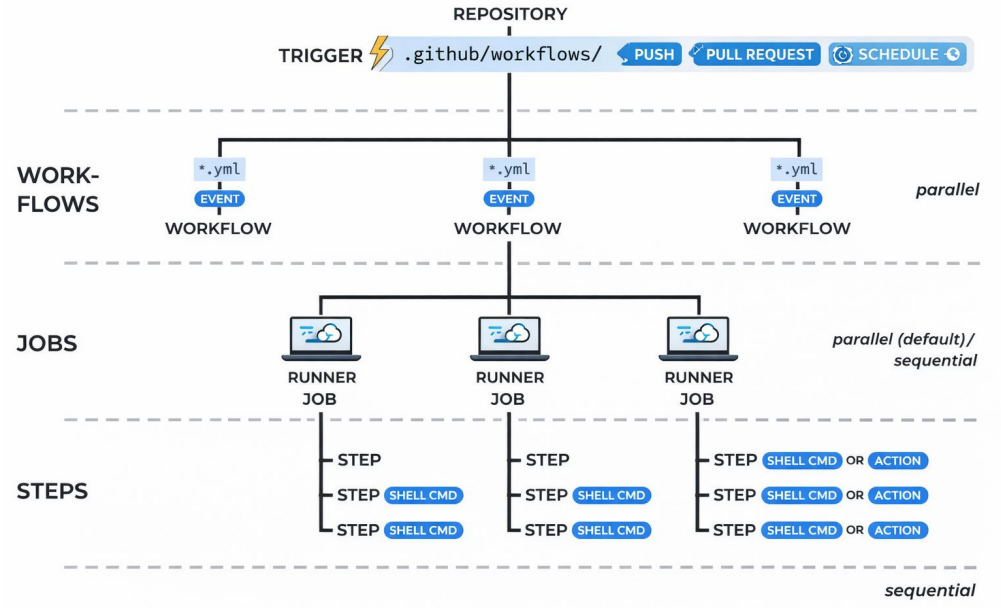
```yaml
services:
  postgres:
    container_name: postgres
    image: postgres:${POSTGRES_VERSION:-latest}
    ports:
      - "5432:5432"
    environment:
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=postgres
      - POSTGRES_DB=postgres
    volumes:
      - postgres_data:/var/lib/postgresql/data
    network_mode: host
```

CI/CD

- name
- run-name
- on
  - push, pull_request, …
- branches
- paths
- paths-ignore
- workflow_dispatch
- input
- workflow_run
- workflow_call
- jobs
- runs-on
- if
- steps
- uses
- with
- environment
- concurrency
- defaults

CI/CD

- env
- run
- working-directory
- shell
- container
  - image
  - env
  - port
  - …



CI/CD

Categoria	Elementos	Onde usar	Observação
Configuração global	env, defaults, permissions	workflow, job	Herdam para níveis abaixo
Execução (infra)	runs-on, needs, jobs	apenas em job	Define onde e como o job roda
Execução granular	run, uses, with	apenas em step	Define comandos e actions
Entrada de dados	inputs, with	inputs → workflow with → step	Contexto depende do uso
Controle de fluxo	if	job e step	Condicional de execução

Exercício 1

- https://github.com/profdiegocbcastro/escape_room_git_3
- Infra Comprometida
  - Tarefa 1
    - Nosso primeiro objetivo é encontrar a flag perdida no git
    - Crie uma branch com os commits que criam as palavras perdida
    - Use echo "valor encontrado" | base64 -d para descriptografar os valores dos arquivos

Tarefa 1

```
git log --all --oneline
d00158f (HEAD -> main, origin/main, or
d90b7c9 pipeline de ci
c9a4d5d cleanup
2f807d5 refactor: limpeza
3da4190 fix: ajuste importante
8a38589 feat: início misterioso
79b9bbb correção da pista da api
9a2ee74 api com dockerfile errado
```

Listar todos os commits

- Outros comandos que poderiam ajudar
  - git log --all -p
  - git reflog
  - git ls-tree -r num_commit
  - git show num_commit
  - git log --name-only
  - git log --name-status

Tarefa 1

```
git log --diff-filter=D --summary
commit c9a4d5dc882deaeed5479f0d64876fd8cac9ca5c
Author: Diego <diegocbcastro07@gmail.com>
Date:   Sat Apr 4 23:13:49 2026 -0300

    cleanup

 delete mode 100644 parte1.txt
 delete mode 100644 parte2.txt
 delete mode 100644 parte3.txt
```

```
Date:   Sat Apr 4 23:13:49 2026 -0300

    cleanup

D       parte1.txt
D       parte2.txt
D       parte3.txt

commit 2f807d5fe8c306fa390ae00c480d4ae21fee3e11
Author: Diego <diegocbcastro07@gmail.com>
Date:   Sat Apr 4 23:13:38 2026 -0300

    refactor: limpeza

A       parte3.txt

commit 3da41909d3230dd8240f4425139091f19ee145cb
Author: Diego <diegocbcastro07@gmail.com>
Date:   Sat Apr 4 23:13:27 2026 -0300

    fix: ajuste importante

A       parte2.txt

commit 8a38589714b7782cbd7c9648de34d1fb18ac604e
Author: Diego <diegocbcastro07@gmail.com>
Date:   Sat Apr 4 23:13:18 2026 -0300

    feat: início misterioso

A       parte1.txt
:
```

```
echo "ZXN0dWRvYmFzdGFudGU=cGFyYQ==" | base64 -d
estudobastantepara
```

```
git checkout -b palavras_git
Switched to a new branch 'palavras_git'
```

```
git cherry-pick 8a38589
[palavras_git 8dc85c6] feat: início misterioso
 Date: Sat Apr 4 23:13:18 2026 -0300
 1 file changed, 1 insertion(+)
 create mode 100644 parte1.txt
git cherry-pick 3da4190
[palavras_git 66b2fd3] fix: ajuste importante
 Date: Sat Apr 4 23:13:27 2026 -0300
 1 file changed, 1 insertion(+)
 create mode 100644 parte2.txt
git cherry-pick 2f807d5
[palavras_git b372523] refactor: limpeza
 Date: Sat Apr 4 23:13:38 2026 -0300
 1 file changed, 1 insertion(+)
 create mode 100644 parte3.txt
```

```
git show c9a4d5dc882deaeed5479f0d64876fd8cac9ca5c
commit c9a4d5dc882deaeed5479f0d64876fd8cac9ca5c
Author: Diego <diegocbcastro07@gmail.com>
Date:   Sat Apr 4 23:13:49 2026 -0300

    cleanup

diff --git a/parte1.txt b/parte1.txt
deleted file mode 100644
index 28e5769..0000000
--- a/parte1.txt
+++ /dev/null
@@ -1 +0,0 @@
-ZXN0dWRv
diff --git a/parte2.txt b/parte2.txt
deleted file mode 100644
index 0c5186a..0000000
--- a/parte2.txt
+++ /dev/null
@@ -1 +0,0 @@
-YmFzdGFudGU=
diff --git a/parte3.txt b/parte3.txt
deleted file mode 100644
index cc0ce77..0000000
--- a/parte3.txt
+++ /dev/null
@@ -1 +0,0 @@
-cGFyYQ==
```

Tarefa 1

```
git log --oneline palavras_git --not main
b372523 (HEAD -> palavras_git) refactor: limpeza
66b2fd3 fix: ajuste importante
8dc85c6 feat: início misterioso
```

Exercício 1

- Tarefa 2
  - Use o shell para subir o dockerfile
  - Coloque a aplicação para funcionar pelo dockerfile
  - Descubra a flag que se encontra dentro da API
  - Encontre as duas flags que estão dentro do container
- Tarefa 3
  - Faça a CI/CD funcionar
  - Recupere a última parte que falta para montar nossa frase

Tarefa 2

```
docker image ls
REPOSITORY   TAG       IMAGE ID       CREATED          SIZE
ctf          latest    8f93634f6655   2 minutes ago    1.1GB
docker image inspect ctf
```



Palavras: Estudo bastante para tenham

Tarefa 2

```
FROM node:18
# TODO: algo está faltando aqui

COPY package.json .
RUN npm install

# TODO: algo está faltando aqui

EXPOSE 3000

ENV SECRET_DATA="116 111 100 111 115"

RUN echo $SECRET_DATA | awk '{for(i=1;i<=NF;i++) printf "%c",$i}' > /tmp/word

RUN echo -n $(cat /tmp/word) | md5sum | awk '{print $1}' > /tmp/hash

ARG FINAL_FLAG

LABEL ctf.flag=$FINAL_FLAG

CMD ["node", "index.js"]
```

WORKDIR /app

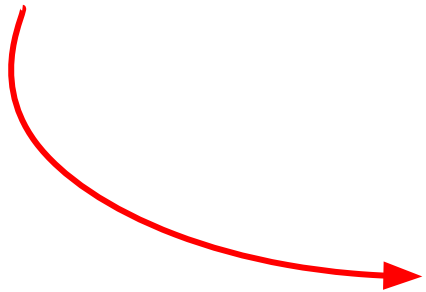
COPY . .

Palavras: Estudo bastante para tenham

```
docker image ls
REPOSITORY   TAG       IMAGE ID       CREATED              SIZE
ctf          latest    24efa1bc5a16   About a minute ago   1.1GB
docker run -it --rm ctf
```

```
docker create --name temp ctf
f54d1d24446daa395bec256a7e6e962d4d5a8061b0bb1ca496d9957a67a057f2
```

```
docker cp  temp:tmp/word .
Successfully copied 2.05kB to /home/diego/Documentos/escape_room_git_3/app/.
```



dockerfile M | word U

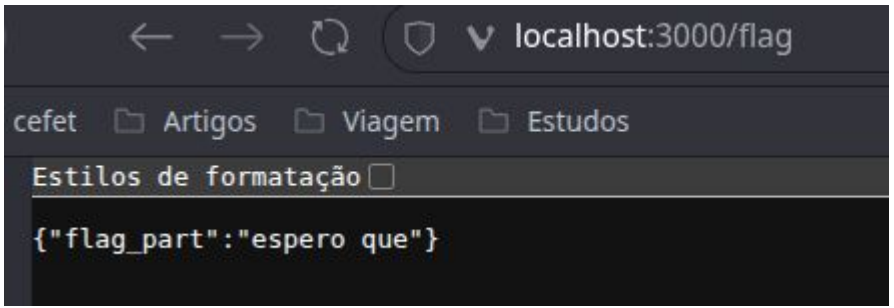
app > word

1 todos

Palavras: Estudo bastante para tenham todos

Tarefa 2

```
docker run -p 3000:3000 -it --rm ctf
Servidor rodando em http://localhost:3000
```



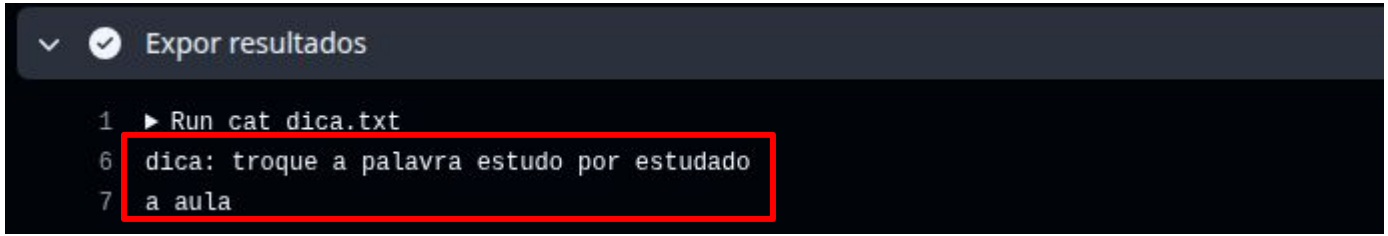
Palavras: Estudo bastante para tenham todos espero que

Tarefa 2

```yaml
- name: Instalar dependências
  run: npm install
  working-directory: ./app

- name: Usar variável secreta
  run: |
    echo "Deploy token: ${{ secrets.RENDER_TOKEN }}"

- name: Deploy (fake)
  run: curl -X POST ${{ secrets.RENDER_DEPLOY_HOOK }}
```



Palavras: Espero que todos tenham estudado bastante para a aula

Exercício 2

- Um colaborador estava trabalhando em um projeto importante e, após várias alterações, fez um commit. Porém, percebeu que havia incluído um arquivo sensível por engano.
  - Desafio 1: Ele precisa voltar o último commit sem perder as alterações feitas.
  - Ao tentar corrigir, ele percebe que já havia feito um git stash anteriormente e esqueceu disso.
  - Desafio 2: Recuperar as alterações que estavam guardadas no stash.

Exercício 2

- Desafio 1: Ele precisa voltar o último commit sem perder as alterações feitas.
  - git reset --soft HEAD~1
  - git rm -r --cached .
- Desafio 2: Recuperar a s alterações que estavam guardadas no stash.
  - git stash list
  - Aplicar
    - git stash apply
    - git stash pop
    - git stash apply stash@{1}

Exercício 2

- Um colaborador estava trabalhando em um projeto importante e, após várias alterações, fez um commit. Porém, percebeu que havia incluído um arquivo sensível por engano.
  - Depois disso, ele decide investigar melhor o histórico e descobre que a informação correta estava em um commit antigo.
  - Desafio 3: Encontrar e restaurar um arquivo específico de um commit anterior.
  - Por fim, ao tentar enviar tudo para o repositório remoto, percebe que sua branch está desatualizada em relação à main.
  - Desafio 4: Atualizar a branch corretamente antes de fazer o push, sem perder suas alterações.

Exercício 2

- Desafio 3: Encontrar e restaurar um arquivo específico de um commit anterior.
  - git log
  - git checkout <hash_do_commit> -- nome_do_arquivo
  - ou
  - git restore --source=<commit> nome_do_arquivo
- Desafio 4: Atualizar a branch corretamente antes de fazer o push, sem perder suas alterações.
  - git pull origin main
  - git pull --rebase origin main
  - git push origin nome-da-branch

Exercício 3

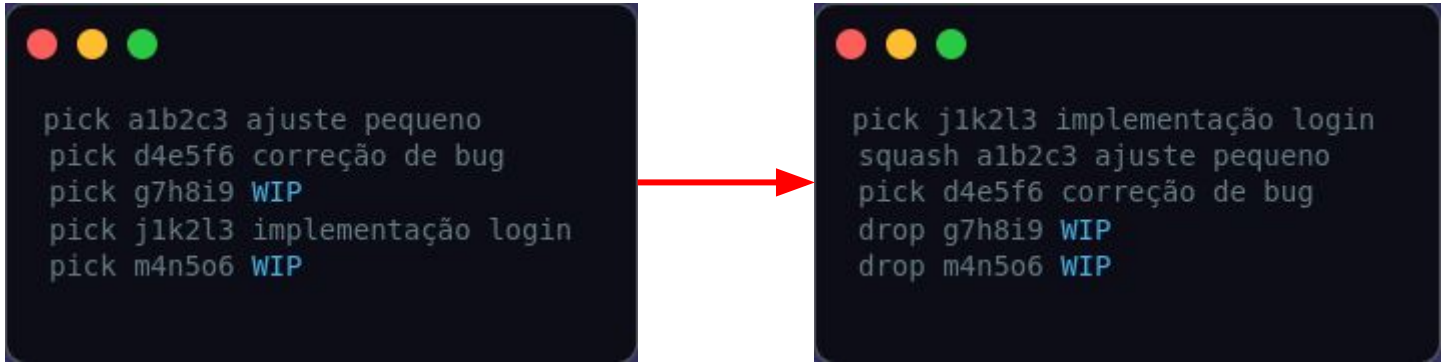
- Um colaborador fez vários commits na branch feature/login, mas o histórico ficou bagunçado:
  - a1b2c3 - ajuste pequeno
  - d4e5f6 - correção de bug
  - g7h8i9 - WIP
  - j1k2l3 - implementação login
  - m4n5o6 - WIP

<novo_hash> - implementação login (com ajuste pequeno)
<novo_hash> - correção de bug

- Ao revisar, ele percebeu que:
- Os commits “WIP” não fazem sentido ficar no histórico
- O commit “ajuste pequeno” deveria ser juntado com “implementação login”
- A ordem dos commits está confusa (implementação deveria vir primeiro)

Exercício 3

git rebase -i HEAD~5



git rebase --continue

Exercício 4

- https://github.com/profdiegocbcastro/gerencia-de-config
  - Pasta: APIs para exercícios
  - Coloque a API feita em python para rodar no docker
  - Crie um docker compose para rodar ela e um banco de dados postgreSQL
  - Crie uma pipeline CI/CD
    - build
    - docker build
    - Deploy no render

Exercício 4

```dockerfile
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 3000

CMD ["python", "app.py"]
```

```yaml
services:
  db:
    image: postgres:15
    container_name: alunos_db
    restart: always
    env_file:
      - .env
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

  api:
    build: .
    container_name: alunos_api
    restart: always
    ports:
      - "3000:3000"
    env_file:
      - .env
    depends_on:
      - db
    volumes:
      - .:/app

volumes:
  postgres_data:
```

```yaml
name: CI-CD API Python

on:
  push:
    branches:
      - main

jobs:

  build:
    runs-on: ubuntu-latest

    defaults:
      run:
        working-directory: "APIs para exercícios/api-python"

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Instalar dependências
        run: pip install -r requirements.txt

      - name: Verificar build
        run: |
          python -m py_compile app.py
```

```yaml
docker:
  runs-on: ubuntu-latest
  needs: build

  defaults:
    run:
      working-directory: "APIs para exercícios/api-python"

  steps:
    - name: Checkout
      uses: actions/checkout@v4

    - name: Login Docker Hub
      uses: docker/login-action@v3
      with:
        username: ${{ secrets.DOCKERHUB_USER }}
        password: ${{ secrets.DOCKERHUB_PWD }}

    - name: Build da imagem
      run: |
        docker build -t ${{ secrets.DOCKERHUB_USER }}/alunos-api:latest .

    - name: Push da imagem
      run: |
        docker push ${{ secrets.DOCKERHUB_USER }}/alunos-api:latest

deploy:
  runs-on: ubuntu-latest
  needs: docker
  steps:
    - name: Deploy via hook Render
      run: |
        curl -X POST ${{ secrets.RENDER_DEPLOY_HOOK }}
```